The runtime driver contract (any language)

TBK SPEC 17:2026, Version 1.0

The Tebako Project

Unrestricted
November 01, 2026
TBK SPEC 17:2026, Version 1.0
Tebako
© 2026 The Tebako Project. All rights reserved

Ribose

Contents

Foreword 4

1. Invocation surface 4

2. Environment 7

2.1 The `TEBAKO_TFS_MOUNTS` grammar (the preload re-mount wire form) 8

2.2 Interpreter-option env (`interp_env`, IMPLEMENTED 2026-10-07 — tebako#559) 9

2.3 The trust bridge (roadmap 81, IMPLEMENTED 2026-09-12) 10

3. File IO semantics 11

4. Exit codes 11

5. Provenance 12

6. Implementation and contract version 12

7. The windows materialize tier (locked 2026-09-17) 12

7a. The invoked program name (tebako#237) 14

7b. Co-mounting run-time-resolved payloads (tebako#460) 15

8. The min-runtime floor (spec 03 §2.9, tebako#666) 15

© 2026 The Tebako Project

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized otherwise in any form or by any means, electronic or mechanical, including photocopying, or posting on the internet or an intranet, without prior written permission. Permission can be requested from the address below.

The Tebako Project

Suite 1, 8/F, New Henry House
10 Ice House Street
Central
Hong Kong

copyright@ribose.com
www.ribose.com

© 2026 The Tebako Project

Foreword

Normative contract between a tebako runtime (the interpreter binary a runtime payload provides) and the tebako loader/dispatcher. Language- agnostic: ruby is the first implementation; python/julia follow this exact contract (roadmap 22's "add a language" playbook). The driver executes LINKED into the interpreter exe (factory-built runtimes) or as the WRAPPER exe of spec 29 (repacked runtimes — upstream bytes tebako does not compile); the wire is identical, and the launcher ABI never learns which pattern a runtime uses.

1. Invocation surface

FIGURE 1

```
<runtime> --tebako-image <self|image-path>:<slot|->:<mount> .
..
          [--tebako-trace <host-path>]
          --tebako-entry <argv0> <user args...>
```

- `--tebako-image` triples mount payloads BEFORE the interpreter starts: `<self>` = read the image from the running executable's own tpkg slot `<slot>`; `<image-path>` = read from a file. Slot tokens: a bare file (no trailer) mounts whole — slot `0` ≡ `-`; a packaged file mounts the numeric slot's trailer-described region, and `-` on a packaged file is a named error. A runtime-role slot (`format_id = TPKG_FORMAT_RUNTIME`) is never mounted. (The image-era `TEBAKO_RUNTIME_IMAGE` case is a bare path mounted whole — the `-` semantics without a triple.)

© 2026 The Tebako Project

- **Mount order:** the env image first — on a runtime-on-runtime boot (spec 33) the OWNER's env image, immediately followed by the DEPENDING runtime's env image as the first triple — then payload triples in argv order; the table is longest-prefix and nested mounts are legal; any failure unmounts everything — never a partial mount. A mount whose ancestors exist in no mounted image MATERIALIZES them as synthesized read-only directories: `stat`/`readdir`/`realpath` walk the boundary (a `gem-home` slice's point dir enumerates its mounted children with plain `Dir`, and `Gem.paths=`'s component-wise realpath resolves each mounted home); a readdir at a boundary merges the image's own entries with the mounted-children names, the image's entry winning a name collision. The synthesis is read-path only — it writes nothing and changes no image.
- **The uniform VFS namespace (locked 2026-08-06):** declared mount points are POSIX absolute paths on every platform — in manifests, in trailer slot records, and on this wire. On windows the namespace presents on its own drive: the driver qualifies every declared mount `<mount>` onto the runtime root's drive (`<drive><mount>`) before any mount, union, or entry computation. The runtime root is a per-platform baked default owned by the runtime factory (ruby: `/__tfs__` on POSIX, `A:/t` on windows — short by owner decision, MAX_PATH headroom on every in-image path). An interpreter's C-level path expansion re-roots drive-relative paths (`/...`) onto the process cwd drive; only drive-qualified paths are stable across expansion, so qualifying is what keeps payload paths inside the VFS. The wire grammar therefore never carries a drive letter: `<mount>` is always the declared form, and a declared mount naming a drive is malformed. Lexical normalization inside the VFS treats a drive prefix as absoluteness itself: `A:/` is the drive ROOT — it normalizes to `A:/`, never to the relative-looking `A:` no mount holds (a collapse would drop the query off the dispatch table, and a sibling mount at `A:/t` would answer in the drive root's place).
- **Run-time root override (`TEBAKO_MOUNT_ROOT`, locked 2026-08-08):** the baked root is the default, never the only spelling. When `TEBAKO_MOUNT_ROOT` is set in the runtime's environment, the driver mounts the env image at that root instead and reports it from `tebako_mount_point` (the io-routing patches and the interpreter's rbconfig follow — era-2 rbconfig emits `ENV["TEBAKO_MOUNT_ROOT"] || <baked>`). The override is validated before any mount — an absolute path (`/...` or drive-qualified `X:/...`), no trailing slash, no `..` — a malformed value is exit 65 naming the variable. The override is then gated post-mount on the env image's layout grant (`mount_root_override: true`, layout schema_minor 1): an image without the grant predates the override era (its rbconfig is pinned to the baked root), so the driver refuses with exit 78 naming both the override and the image — never a boot whose load paths point at an unmounted root. Declared payload mounts qualify onto the override's drive exactly as they do the baked root's.
- **Mount modes (locked 2026-08-04):** every mount is `exclusive` (default) or `union`, declared per slot in the package manifest's `mounts:` block (spec 03 §6). An exclusive mount onto an occupied point is a named error (EEXIST). A union mount onto an occupied point merges the trees: directories combine, file conflicts resolve by the declared precedence (`after-env` — over the env image — or `after:<slot>`); union members are read-only and the union set (point + members + precedence) is journaled at boot. The driver reads the mode from the running package's OWN trailer (the `<self>` manifest block) — mount semantics never ride the argv grammar, so the

© 2026 The Tebako Project

launcher ABI is unchanged and drivers predating this section refuse a union package loudly (EEXIST), never silently. Payloads handed over without a package manifest (shim dispatch, bare images) are always exclusive.

- **Carried and shared payloads are indistinguishable on this wire** (spec 23 §13, additive 2026-08-25): a SHARED slice the loader resolved into the machine cache is handed over as an ordinary `<image-path>:-:<mount>` triple — a bare cache file, mounted whole; a CARRIED slice is the `<self>:<slot>:<mount>` form. Mount order, longest-prefix dispatch, union/exclusive modes, and unmount-all-on-failure apply identically — the driver neither knows nor cares where the bytes physically live. Resolution is loader-side (spec 05 §5): an unresolvable shared slice is the loader's named error BEFORE the handoff, never a driver case; and resolution follows the press-time lock (spec 23 §4) by locked digest, never fresh semver.
- `--tebako-entry` separates loader args from user args; `argv0` is the entrypoint inside the mounted tree, resolved against the FIRST `--tebako-image` mount (the app payload) — or against the runtime root when no image spec is given. On a runtime-on-runtime boot (spec 33) the FIRST triple is the depending runtime's env image: the entry resolves against the first triple whose mounted manifest is not that env image — with no payload triples, against the depending runtime's own mount (the self-boot smoke form) — and a bare NAME resolves against the DEPENDING runtime's `provides.entrypoints`, not the env image's. The driver verifies the entry's presence against the mounts the boot itself established (named error 65 when absent); an entry outside them belongs to the interpreter's own startup. A bare NAME (no `/`) is the keyword form. `self` is the reserved interpreter keyword: the boot starts the interpreter itself with the user's args and drops the keyword (the deploy shims' re-entry form). Any OTHER bare name (spec 30 §2's spawned-dependency dispatch) resolves against the ENV image's own `provides.entrypoints` (spec 03 §2.2's runtime block): the declared `args_default` composes exactly as an app entry's would and the declared path is verified against the mounts THIS boot established; an undeclared name is the named error 65. With images but NO `--tebako-entry` at all, the boot mounts and starts the interpreter with its own args (the smoke form).
- `--tebako-trace <host-path>` (spec 25 §2, additive): arm the interception trace bus at boot, BEFORE any mount — the channel file is opened once and appended for the process's life. The TEBAKO_TRACE env names the same channel; the argument wins when both are set. A channel that cannot be opened is a loud stderr note and a disarmed bus, never a boot failure (spec 25 law 1: observability never gates). The consumed flag never reaches the interpreter's argv.
- The interpreter's own args are skipped (and `--tebako-extract` stays theirs); an unknown `--tebako-*` flag is a named error, never silently ignored.
- Everything before the first `--tebako-*` is the loader's; everything after `--tebako-entry` is the user's, verbatim. On success the driver replaces the process argv with `[<original argv0>, <args_default…>, <entry resolved in the VFS>, <user args…>]` — the entrypoint's declared `args_default` (spec 03 §2.2), read from the app payload's own manifest, composes between the interpreter and the entry, and the program name stays at index 0 so the interpreter parses its argv conventionally and takes the entry as its script. args_default has ONE owner (tebako#503, 2026-08-31): the runtime side — this rewrite, linked and wrapper

© 2026 The Tebako Project

patterns alike (spec 29 §1). Loaders (the shim's dispatch, the bootstrap's handoff) never carry it; a zero-runtime entrypoint (no driver exists) is the exception — there the shim appends them as the program's leading args.

2. Environment

TABLE 1

var	meaning
`TEBAKO_RUNTIME_IMAGE`	image-era: absolute path of the runtime's own `.tfs` (driver mounts it instead of any embedded image)
`TEBAKO_SPAWN_LOCK`	spec 30 §2: the dispatcher's spawn-time pins for the payload's `kind: runtime` edges — `engine=language_version:tebako_version`, `;`-joined in manifest order. The driver consumes it at spawn (cache-only resolution to the pinned pair) and never propagates it to a spawned child
`TEBAKO_TFS_MOUNTS`	`image[:slot]:mount,…` — mounts to establish (preload-shim path); grammar in §2.1
`TEBAKO_JAIL`	jail policy env form (spec 08) — the driver/preload enforces it
`TEBAKO_JAIL_SOURCE`	audit label of the policy's origin (`manifest` / `user` / `manifest+user`, or the exporting surface) — journaled with every denial (spec 08 §2)
`TEBAKO_JAIL_JOURNAL`	explicit audit-journal path (default: `$TEBAKO_HOME/journal.log`)
`TEBAKO_TRACE`	spec 25 §2: the interception trace bus's channel file (JSONL). Opened at boot before any mount, appended for the process's life, never policy-gated; children re-derive it from the inherited env. `--tebako-trace` wins when both are set
`TEBAKO_EXEC_CACHE`	spec 22 §6: the boot's exec-cache root — materialized binaries/libraries live under it (per-process on POSIX; leave-in-place and content-keyed on windows, spec 22 §2.1); read-only to payloads
`TEBAKO_RUNTIME_DLL`	spec 22 §2.1 (windows only): the runtime's own PE module basename (e.g. `x64-ucrt-ruby340.dll`), flowed from the factory record — the single owner — and exported by the driver at boot. The tfs PE closure walk excludes a bare import name matching it (case-insensitive, bare names only); POSIX legs never read it
`TEBAKO_PRELOAD_SHIM`	spec 22 §3: the preload shim's in-VFS path, flowed from the env image's `preload_shim` layout grant — the interpreter's spawn hook reads it (never a hand-written copy); the driver additionally arms `LD_PRELOAD` (ELF) / `DYLD_INSERT_LIBRARIES` (macOS) with the materialized host copy
`TEBAKO_MOUNT_<SLUG>`	spec 22 §6 + v2-1/20: per co-mounted payload image, its physical mount point (drive-qualified on windows). SLUG is the mount's mechanical uppercase form: `/tools/inkscape` → `TEBAKO_MOUNT_TOOLS_INKSCAPE`; two mounts slugging alike is a named

© 2026 The Tebako Project

var	meaning
	boot error (65). The root mount `/` exports nothing — `TEBAKO_MOUNT_ROOT` stays the mount-root override (§1). Under the §7 materialize tier the value is the extracted HOST dir, never the VFS point
`TEBAKO_MATERIALIZE_BOOT`	spec 17 §7 (windows only): the materialize tier's respawn marker, exported together with the rewired `TEBAKO_MOUNT_ROOT`; a boot finding it scrubs both before §1's override is read and re-derives the tier from the env image's grant
`PATH`	spec 22 §3.2: led by the launcher dir (`<exec-cache-leaf>/wrap-bin/`) when the env image delivers the preload shim — every declared dependency executable materialized as a self-injecting wrapper (unix; the SIP-strip answer) — then every co-mounted DEPENDENCY image's declared bin dirs (the dirname of each `provides.entrypoints[].path` / `provides.executables[].path` in the image's own `/__tpkg__/manifest.yaml`, joined under its mount, in triple order). The first triple (the app payload) never contributes; an image without a readable manifest declares no bins; a corrupt manifest or an unmaterializable declared executable is a named 65. On windows the boot-materialized library-alias directories complete the same lead (spec 22 §2.1's bare-name rule) — every co-mounted image contributing, the env image and the app payload included; the lead order is locked: launcher dir → dependency bin dirs → alias dirs → the inherited `PATH`. Under the §7 materialize tier the dependency bin dirs name the extracted HOST trees (`<trees>/<key>/<bin>` — the same mount→host map the `TEBAKO_MOUNT_<SLUG>` export consumes), never the VFS points: the host loader's `PATH` search cannot resolve the VFS spelling
`SSL_CERT_FILE`	spec 22 §4 (the cert convention's env surface — driver-owned, the driver being the single owner of the materialized host path): when a mounted image declares a `materialize:` entry ending `ssl/cert.pem`, the driver exports the cert's materialized HOST path at boot, in both boot shapes. An unset/empty value is set; a value lexically under the effective runtime mount root (a stale in-VFS spelling — `A:/t/ssl/cert.pem` — resolved by the patched IO but unreadable by libcrypto's native CRT IO) is rewritten; a real host path is the user's own configuration and always wins. No declared cert → nothing is set (the POSIX no-op) UNLESS the operator CA store is non-empty (tebako#541, spec 22 §4 Rule R4): then the bundle is the enumerated platform store plus the store PEMs under the exec cache's `resources/store-trust/` namespace, and this var names it. When the trust bridge (§2.3) is in force the exported path names the MERGED bundle, content-keyed by the merge inputs

2.1 The `TEBAKO_TFS_MOUNTS` grammar (the preload re-mount wire form)

FIGURE 2

```
mounts     = entry *( "," entry )
entry      = image-ref ":" mount
image-ref  = image-path [ ":" slot ]
image-path = absolute host path (may itself contain colons)
```


© 2026 The Tebako Project

```
    slot       = 1*DIGIT / "-"
    mount      = "/" *path-char              ; a VFS-absolute
    mount point
```

- **Parse from the right.** Per entry: the mount is the field after the last `:` (it MUST start with `/`); the slot is recognized ONLY when the remainder's rightmost field is all digits or exactly `-` — then the image path is what is left. Anything else after a colon stays part of the image path: host paths may contain colons, and the grammar never rejects them.
- **Slot meaning.** Absent ≡ `-` ≡ the whole file (a bare image — the pre-slot behavior, unchanged). A numeric slot names a package slot: the consumer resolves slot → byte region against the file's tpkg trailer (spec 04). On a trailer-less file a numeric slot is a named error; a slot out of range is a named error; a runtime-role slot (`format_id == 4`) is never mounted — a named error.
- **Windows.** Right-parsing keeps drive colons intact: `C:\pkg.tebako:0:/__tfs__` is package `C:\pkg.tebako`, slot 0, mount `/__tfs__`; `C:\image.tfs:/data` is the bare image `C:\image.tfs`, whole file, mount `/data`.
- **No `<self>`.** The CLI triple form's `<self>` token (§1) is NOT meaningful in the env form — the consuming process is a different executable from the one that was stitched. The emit side always spells the package's absolute host path.
- **Emit rule.** A mount established from a whole bare image serializes as `image:mount`; a mount established from a package slot serializes as `image:slot:mount`. Consumers re-exporting their mount table to a spawned child MUST preserve the slot form, so the child mounts the same region — never the whole package file (mounting a packaged file whole sniffs the trailer and fails; spec 22's spawn re-entry is the victim).
- **Union serialization.** A union mount (§1's `mode: union`) serializes as MULTIPLE entries at the same mount point: the incumbent first, then each member in shadow order (the order they were unioned on). A consumer that can union (the preload shim) MUST treat a repeated mount point as a union member — layered over the earlier declaration with this spec's §1 semantics — never as a duplicate-point error. A consumer that cannot union MUST fail closed with a named error. Serializing only the incumbent is a wire bug: the child boots with half the tree (the packed-mn POSIX failure — the app payload unioned over the env image at the runtime root never reached spawned children).

2.2 Interpreter-option env (`interp_env`, IMPLEMENTED 2026-10-07 — tebako#559)

The spec 07 §9.1 chain's product is ordinary process env, not a driver wire var: the dispatcher (the shim in managed mode, the bootstrap in standalone) computes the effective interpreter-option map and exports each winning key BEFORE the exec/handoff, so the interpreter's own option processing reads it at boot (`RUBY_YJIT_ENABLE`, `PYTHON_JIT`, `JAVA_TOOL_OPTIONS`, ...). Consequences on this contract:

© 2026 The Tebako Project

- **The driver applies nothing and never touches these vars.** They are not TEBAKO_* control vars, not in the M7 scrub list, and not rewritten by any boot pass — a value the driver finds set is the chain's outcome by construction.
- **Boot order is safe by construction:** the driver's boot exports (the §2 table's own vars) and the materialization passes all run before the interpreter initializes, and none of them name an interp_env key (the manifest grammar forbids TEBAKO_* there — spec 03 §2.7).
- **Spawned children (specs 30/32)** inherit the parent process's environ, so a spawned entrypoint sees the parent's effective map. A spawned PROVIDER's own L1 `interp_` env does NOT apply on that path in this phase (the driver merges nothing) — that axis is a later amendment; a payload needing it is dispatched through the shim, which runs the full §9.1 chain.
- The auditability requirement is served at dispatch: the effective map and each key's provenance layer render through the spec 15 §4 surface; no per-run journal entry exists in this phase.

2.3 The trust bridge (roadmap 81, IMPLEMENTED 2026-09-12)

The loader plane's network configuration (spec 04's `network:` grammar — the single resolution owner) already trusts the enterprise proxy story for its OWN fetches: `tls_roots: platform` / TEBAKO_TLS_PLATFORM_ROOTS (the OS store, where the GPO/MDM-pushed proxy CA lives) and additive `extra_ca:` / TEBAKO_EXTRA_CA (org PEMs parsed into the bundled roots). This subsection is the bridge that makes the RUNTIME's TLS stacks follow the same resolution — one config, every plane; the canonical runtime images stay pristine (trust material flows at run time, never a per-org rebuild).

- **The wire.** The dispatcher (shim / bootstrap) resolves netconfig ONCE per process and conveys the verdict to the driver by exporting TEBAKO_TLS_PLATFORM_ROOTS / TEBAKO_EXTRA_CA into the handoff env — env inheritance is the wire (spec 00 §10: no second hand-written copy, no second config read in the driver).
- **The PEM planes (ruby / python / everything openssl- or curl-fashioned).** When either var is in force, the driver's cert convention (§2's SSL_CERT_FILE row, spec 22 §4) materializes a MERGED bundle instead of the image's bare `cert.pem`: the image roots
  - — the enumerated platform store (rustls-native-certs; the driver is

    not size-gated) in platform mode, or the image roots + the parsed `extra_ca` PEMs in additive mode. The merged bundle lives under the same resources cache with the same digest discipline (spec 22 §4's write-once + per-boot rehash), content-keyed by the merge inputs. Precedence is the §2 row's unchanged: a user's real host-path SSL_CERT_FILE always wins; the stale in-VFS spelling is still rewritten. Platform + additive simultaneously is the netconfig layer's named error, never the driver's.
- **The operator CA store (tebako#541; spec 22 §4's Rule R4 owns the mechanics).** The machine-wide `$TEBAKO_HOME/trust/ca/*.pem` store merges into the same bundle WITHOUT a conveyed verdict too — an image cert + the store merges when no verdict

© 2026 The Tebako Project

exists; no image cert (the POSIX case) + a non-empty store builds the bundle from the enumerated platform store plus the store PEMs. The wire is unchanged (the driver reads the store itself — it is not size-gated; the loader plane conveys nothing new). The same fail-closed rule covers a corrupt store file.

- **The java plane.** The JVM is a spawned child (spec 30), not a driver boot — its bridge rides the §2.2 interp_env chain on the dispatcher side: in platform mode on windows the chain additively appends `-Djavax.net.ssl.trustStoreType=Windows-ROOT` to `JAVA_TOOL_OPTIONS` (the JVM trusts the OS store natively; additive merge, never a stomp). The `extra_ca`-on-java shape (a materialized PKCS12 truststore + `-Djavax.net.ssl.trustStore`) is the open sub-item of roadmap 81 — until it lands, `extra_ca` with a spawned java edge in force yields a loud journal line naming the gap, never a silent partial-trust boot.
- **Fail closed.** A malformed `extra_ca` PEM, an unenumerable platform store, or an unwritable merge destination is a named boot error (65 class) — never a silent fall back to the image bundle alone, which would surface as inscrutable TLS errors deep in the payload. There is no verify-off spelling.

3. File IO semantics

The runtime's IO MUST route mounted paths through the TFS layer (`tebako_fs_*` or an equivalent in-process VFS): read-only payload images; host paths pass through subject to the jail policy; writes to payload images fail EROFS. Two implementation patterns (spec 07 §8): patched-interpreter io-routing (ruby) or the preload interposition shim (unmodified dynamic binaries).

4. Exit codes

The runtime preserves the loader's named codes (65–74) when the failure is loader-side; runtime-side failures use the interpreter's own codes. `--tebako-extract` is the runtime-side option riding the user-arg passthrough: dump mounted images to disk and exit 0.

© 2026 The Tebako Project

5. Provenance

A runtime payload's manifest (spec 03) declares `provides` {engine, version, abi_line, platform} + `built_from` — the dispatcher's compatibility check consumes exactly these fields.

6. Implementation and contract version

The reference implementation is `crates/tebako-driver` (Rust, staticlib * rlib) — the v1 C++ `tebako-main` driver is retired. The driver executes in one of two patterns: LINKED into the interpreter exe (factory-built runtimes — the reference form) or as the WRAPPER exe defined in spec 29 (repacked runtimes); both patterns declare the same contract. Runtimes whose driver implements this document's widened grammar (image-path triples, bare-file slot tokens, env-image-first multi-mount, direct `--tebako-entry` execution) declare `contract_version 2` in their release manifest (spec 06 §6); the compiled-in constant (`tebako_driver_contract_version()`) and the manifest must agree.

7. The windows materialize tier (locked 2026-09-17)

A runtime whose interpreter CANNOT consume the driver's mounted images — the zero-patch contract: no io-routing patches, and windows has no preload interposition tier (spec 22 §2) — boots on windows from EXTRACTED host trees instead. This section is a boot-time behavior tier, not a wire change: §1's grammar, the mount table, and every verification step are untouched, and POSIX is unchanged — the tier never engages there (POSIX runtimes read the mounts directly, ruby's patched IO being the reference case).

The trigger is a grant, never a heuristic. The runtime's env image manifest (`/__tpkg__/manifest.yaml`, kind `runtime`) declares `provides.windows_boot: materialize` (spec 03 §2.2, `payload-manifest.yaml` schema_minor 11 — old readers ignore the key and keep the mounted-boot behavior, which for such a runtime is its standing named refusal). On windows, after the env image and payload mounts are established and verified exactly as §1 describes, the driver reads that manifest: the grant present selects this tier; absent, the boot proceeds exactly as before.

© 2026 The Tebako Project

Extract. Every mounted image of the boot — the env image first, then each payload triple in order — is extracted as a TREE (not per file; spec 22 §4's class-R per-file discipline is the per-file ancestor of this one) into the exec cache:

FIGURE 3

```
<TEBAKO_EXEC_CACHE>/trees/<tree-key>/        the extracted
image tree
<TEBAKO_EXEC_CACHE>/trees/<tree-key>.tfs-digest   the
verification record
```

- `<tree-key>` is the image's own content key — the store sidecar's sha256 prefix (the exec cache's segregation idiom, spec 22 §6), with the slot number appended (`-<slot>`) when the image is a package region, so two slots of one package never share a tree; a no-store dev boot keys the path, exactly as the cache root itself does.
- **Write-once, tmp+rename, per-entry flock.** Extraction streams the mounted tree into a per-process staging dir (`.<tree-key>.part-<pid>`), hashing in flight (per file the tfs-merkle-1 file construction; the tree digest is the sha256 over the sorted walk's `D <rel>/` / `F <rel> <merkle>` lines), then renames the record into place BEFORE the tree — content without a record is foreign by construction, and a partial tree is never visible at the final path. Installed files are read-only (Rule R3). Concurrent boots serialize on the per-entry flock (120 s timeout, then a named error with the stale-lock hint — the store's spec 05 §4 discipline). The walk lists each image's OWN entries (the image-native listing, not the interpreter-facing mount-merged one): a sibling mount BELOW the walked point — the env image at `A:/t` beside a payload at the drive root `A:/` — extracts into its own tree by its own key, and its boundary name never joins this walk (or the walked image's tree).
- **Digest-pinned reuse.** A cached tree is served only after it re-verifies against its record (the host tree re-walked and re-hashed to the recorded digest). A match is served with zero extraction — the second boot is free. A mismatch, a missing record, or a corrupt record is the cache tampered or corrupt: the tree is wiped and re-extracted ONCE from the (mounted, verified) image; a tree that still fails verification after re-extraction is the named 70 (EX_TEBAKO_SHA), never a silently served corruption. Extraction IO failures mid-tree are named 74s with the staging dir abandoned (never renamed into place). A tree holding a symlink or a special entry is a named 74 — the tier extracts regular files and directories only.
- **The trust chain is untouched.** Images are verified at fetch/install (spec 09) and mounted exactly as before; the tier only changes WHERE the interpreter's bytes are read from. The record pins the cache tree to the bytes the verified image served; the per-boot re-walk pins the tree to the record.

Rewire. With the trees in place the driver rewires the handoff for the interpreter's plain host IO:

© 2026 The Tebako Project

- TEBAKO_MOUNT_ROOT is set to the extracted env-image tree (a drive-qualified host path) — the era-2 rbconfig pattern (ENV["TEBAKO_MOUNT_ROOT"] || <baked>, generalized to any runtime) then resolves every runtime-relative path onto plain host files. This is the tier's own rewiring, NOT a §1 user override: the mount_root_override layout grant does not gate it (the runtime opted in by declaring the tier), and the driver's own mounts were already established at the baked root before the var is set.
- The mount-discovery surface (§2's TEBAKO_MOUNT_<SLUG> table) points at the extracted HOST dirs, not the VFS points — a consumer reading a co-mounted payload's files reads host files, as the interpreter must.
- The §3.2 dependency bin dirs on PATH (spec 22 §3.2) name the extracted HOST dirs too (<trees>/<key>/<bin> per dependency mount, through the same mount→host map the discovery export consumes) — the host loader's PATH search cannot resolve the VFS spelling, so a VFS-spelled lead silently resolved nothing (the xml2rfc windows leg's mingw DLLs, which only a beside-the-pyd staging could reach before this).
- The rewritten argv's entry (§1) resolves to its host path inside the extracted tree of the image that mounted it.
- Respawn. The driver also exports TEBAKO_MATERIALIZE_BOOT=1 with the rewired root. A boot that finds the marker present treats an inherited TEBAKO_MOUNT_ROOT as the tier's own state — both are scrubbed BEFORE §1's override is read (the child's driver mounts at the baked root, re-derives the tier from the env image's grant, and serves the cached trees). Without the marker an inherited root keeps §1's semantics exactly.
- The mounts stay established for the driver's own reads (manifests, spawn planning) for the process's life; the interpreter simply never consults them.

The jail deviation (documented, loud). Host IO under an extracted tree cannot be interposed, so on a windows-materialize boot TEBAKO_JAIL does not confine the payload's reads of host files — there is no VFS on the consumption path to enforce it. POSIX mounts enforce the jail exactly as spec 08 describes. A boot under the tier emits a LOUD notice to stderr naming the deviation, once, before the interpreter handoff.

Exit codes stay inside the loader's named allocation: 65 (a tier declared on a corrupt/lying manifest surfaces through the existing manifest checks), 70 (a tree that fails verification after its one re-extraction), 74 (extraction IO failure, a symlink/special entry in the tree, the flock timeout). Runtime-side failures keep the interpreter's own codes (§4).

7a. The invoked program name (tebako#237)

The driver exports TEBAKO_PROGRAM_NAME on every entry handoff: the NAME the dispatch selected — the bare --tebako-entry keyword verbatim, or the entry path's basename

© 2026 The Tebako Project

(the gem-binstub convention) for the path form. The ruby patch assigns `$0`/`$PROGRAM_NAME` from it at interpreter start, so CLI frameworks that read the program name (the Thor/option-parser class) see the program the user ran, not the resolved in-VFS script path. The variable is a read-once handoff: it sits on the boot-contract side of the credential scrub (spec 03 §2.10) and payloads cannot flip it for children.

7b. Co-mounting run-time-resolved payloads (tebako#460)

A payload slice the boot needs but the package did not embed resolves at first run per spec 05 §4a, then co-mounts EXACTLY like an embedded slot: the loader hands the driver the same `--tebako-image <path>:<slot>:<mount>` triple — the driver cannot tell an embedded slot from a cache-resolved one, and never may (one code path, spec 05 §4a's rule). The resolution happens BEFORE the handoff (bootstrap standalone / shim managed); a named 75-class error (unresolvable), a trust error (digest mismatch), or 65 (a DEPENDS cycle) fires at resolution time — the driver's mount contract is unchanged.

8. The min-runtime floor (spec 03 §2.9, tebako#666)

Every payload manifest the driver reads at boot — the app payload's, a spawned provider's, the env image's own — is gated on the declared `min_runtime_tebako` floor against the runtime's own tooling version (the driver crate version the factory built the exe from; the store grammar names the runtime by it):

- **Clean parse, floor above the runtime:** the boot refuses with the named stale-runtime error — "the runtime is too old for this payload's manifest (needs X, have Y); update the runtime — the payload is not at fault". The key is additive and old-reader-safe, so a manifest can parse fine and still be too new for the runtime's semantics.
- **Parse failure, floor above the runtime:** the manifest almost certainly uses grammar this runtime predates. The driver reads the floor by a tolerant top-level scan of the raw YAML (`PayloadManifest::declared_min_runtime_tebako`) and rewords the refusal to the same stale-runtime message — NEVER "corrupt manifest / self-description lies", which blames the wrong artifact (the tebako#666 failure class: mount-ancestor routes on pre-#616 runtimes, spec-32 `kind: executable` edges on pre-spec-32 runtimes).

© 2026 The Tebako Project

- **No floor, or a satisfied floor, or an unreadable scan:** the existing wording and codes stand unchanged.

The refusal keeps the manifest class's exit code (65, `EX_TEBAKO_MANIFEST`) — the wording, not the code, is the contract change; the loader-side resolution floor (spec 05 §5) is what keeps a stale runtime from being picked in the first place.

© 2026 The Tebako Project